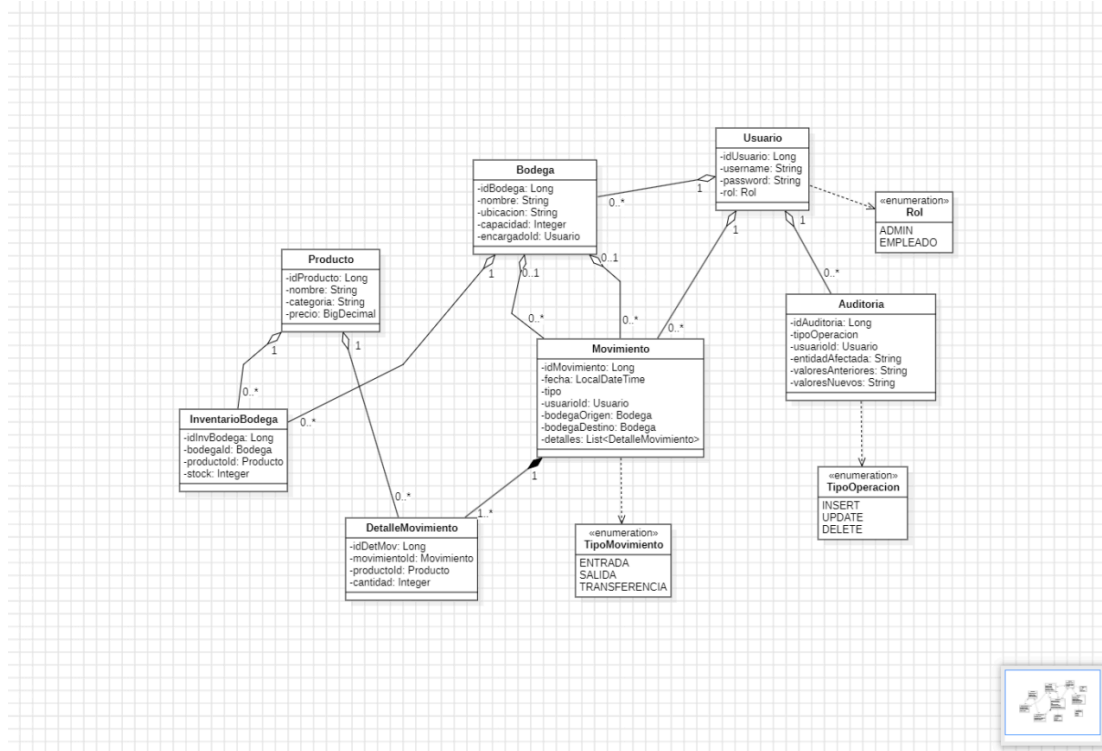


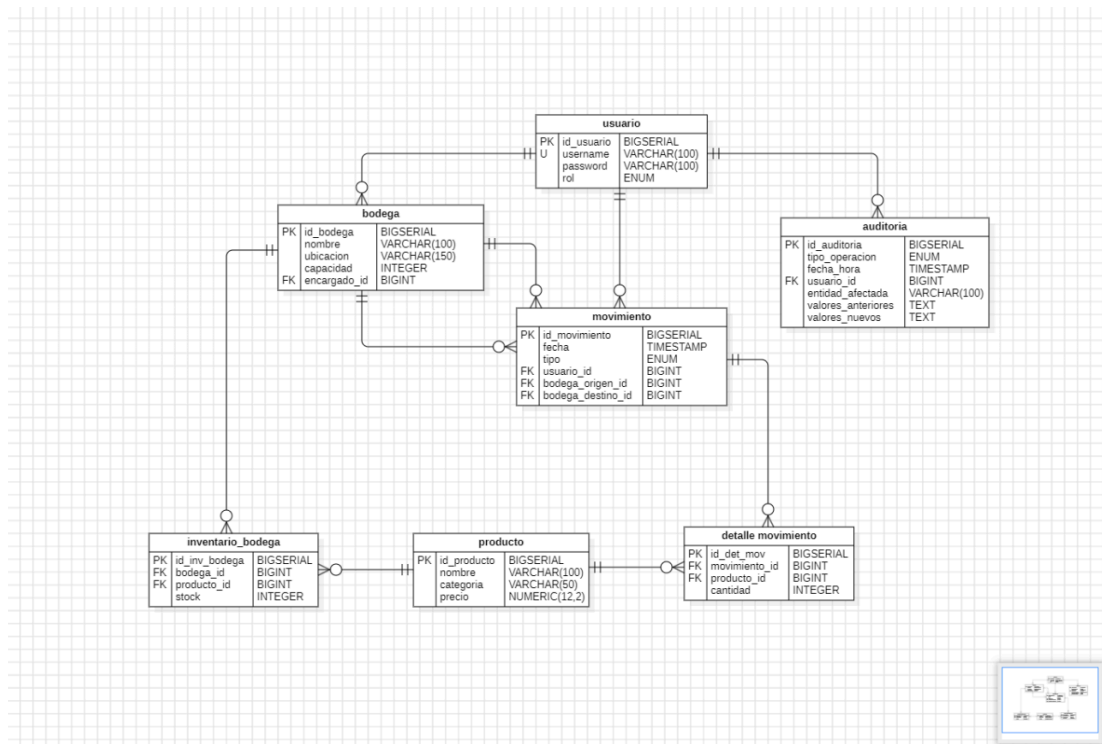
LogiTrack - Documento explicativo

Repositorio LogiTrack: <https://github.com/jorgegmch/logitrack-api.git>

- Diagrama de clases - Hecho en StarUML



- Diagrama ER - Hecho en StarUML



- **Descripción de la arquitectura**

Arquitectura general

LogiTrack API está organizada en capas, siguiendo el enfoque estándar de las aplicaciones Spring Boot. La idea central es que cada capa tenga una sola responsabilidad y que solo se comuniquen con la capa inmediatamente inferior, nunca al revés. Esto permite, por ejemplo, cambiar la lógica de negocio sin tocar el acceso a datos, o reemplazar el motor de base de datos sin modificar los controladores.

El flujo de una petición típica funciona así: el cliente (ya sea Swagger o el frontend propio) envía una solicitud HTTP que llega primero al Controller. El Controller no contiene lógica de negocio; su único trabajo es recibir la petición, validar que los datos de entrada cumplan las reglas básicas (mediante Bean Validation), y delegar el trabajo real a la capa de Service. Ahí es donde vive toda la lógica de negocio: las validaciones más complejas, el control de transacciones, y la orquestación de operaciones que involucran varias entidades a la vez (como registrar un movimiento de inventario, que implica crear el movimiento, sus detalles, y actualizar el stock correspondiente en la bodega). El Service, a su vez, se apoya en los Repository para consultar o modificar datos, los cuales usan Spring Data JPA para traducir esas operaciones en consultas SQL contra la base de datos. Finalmente, las Entity son las clases que representan directamente las tablas de PostgreSQL, mapeadas mediante JPA/Hibernate.

Capas y responsabilidades

Controller: expone los endpoints REST de la API. Recibe las peticiones, valida la entrada con `@Valid`, y llama al Service correspondiente. No toma decisiones de negocio por sí mismo.

DTO: son los objetos que se usan para enviar y recibir datos por la API, separados de las entidades internas del sistema. Hay DTOs de entrada (como `ProductoRequest` o `MovimientoRequest`) y de salida (como `LoginResponse` o `ReporteResumenResponse`), lo que permite que la forma en que el cliente ve los datos no dependa directamente de cómo están modeladas las tablas.

Service: aquí vive la lógica de negocio real. Las validaciones de dominio, el manejo de transacciones con `@Transactional`, y la coordinación de varias operaciones relacionadas ocurren en esta capa.

Repository: interfaces que extienden `JpaRepository`, dejando que Spring Data JPA genere automáticamente las consultas SQL a partir del nombre de los métodos.

Para los reportes de agregación (como el stock total por bodega) se usan consultas JPQL personalizadas con la anotación `@Query`.

Entity: las clases anotadas con JPA que representan las tablas de la base de datos, junto con sus relaciones entre sí (por ejemplo, un producto puede estar relacionado con varias bodegas a través del inventario).

Security: contiene el `JwtService`, encargado de generar y validar los tokens, y el `JwtAuthenticationFilter`, que revisa cada petición entrante, valida el token si existe, y le indica a Spring Security quién es el usuario autenticado.

Config: aquí se configura Spring Security (que rutas son publicas, cuales requieren autenticación, y cuales además requieren un rol específico), la documentación de Swagger, y un componente puente (`ApplicationContextProvider`) que permite que el listener de auditoría, el cual no es gestionado directamente por Spring, pueda acceder a otros componentes del sistema cuando lo necesita.

Exception: centraliza el manejo de errores de toda la aplicación mediante un `@RestControllerAdvice`, transformando cualquier excepción en una respuesta JSON clara y consistente, en vez de dejar que el cliente reciba errores técnicos del framework.

Listener: contiene el `AuditoriaListener`, que se engancha automáticamente al ciclo de vida de las entidades mediante `@EntityListeners` de JPA, y es responsable de registrar cada creación, actualización o eliminación que ocurre en el sistema.

Flujo de autenticación

Cuando un usuario inicia sesión, el sistema recibe su username y password en el endpoint `/auth/login`. Spring Security, a través del `AuthenticationManager`, delega la verificación de credenciales a un `DaoAuthenticationProvider`, que usa el `UsuarioService` (el cual implementa la interfaz `UserDetailsService` de Spring) para buscar al usuario en la base de datos y comparar la contraseña ingresada contra la contraseña almacenada, que está protegida con BCrypt.

Si las credenciales son correctas, el `JwtService` genera un token firmado, que incluye el username, el rol del usuario, y las fechas de emisión y expiración. Ese token es lo que el cliente debe usar en cada petición siguiente. Cuando llega una nueva petición a cualquier endpoint protegido, el `JwtAuthenticationFilter` la intercepta antes de que llegue al controlador, extrae el token del encabezado `Authorization`, verifica que sea válido y que no haya expirado, y si todo está correcto, establece la identidad del usuario en el contexto de seguridad de Spring para que el resto del sistema sepa quien esta haciendo la petición.

Finalmente, la configuración de seguridad define, ruta por ruta y método por método, quién puede acceder a que. Algunas rutas son completamente públicas (como el login), otras requieren solamente estar autenticado, y otras (como la gestión de usuarios o la consulta de auditorías) están reservadas exclusivamente para el rol ADMIN.

Un detalle importante del diseño es que esta autenticación es completamente stateless: el servidor no guarda ninguna sesión. Toda la información necesaria para saber quien es el usuario y que puede hacer viaja dentro del propio token en cada petición.

Auditoría automática

Uno de los requisitos centrales del sistema es que cualquier cambio importante quede registrado automáticamente, sin que el desarrollador tenga que acordarse de hacerlo manualmente en cada operación. Esto se logra con el AuditoriaListener, que está enganchado a las entidades Producto, Bodega, Usuario y Movimiento mediante @EntityListeners.

Este listener aprovecha los eventos del ciclo de vida de JPA: cuando una entidad se carga desde la base de datos, guarda una copia de su estado original en memoria. Cuando esa misma entidad se crea, se actualiza o se elimina, el listener compara el estado anterior con el nuevo, identifica quien es el usuario autenticado en ese momento, y prepara un registro de auditoría con esa información.

Aquí hubo un detalle técnico importante que se resolvió durante el desarrollo: el guardado del registro de auditoría no puede hacerse simplemente dentro de la misma transacción que está modificando la entidad original, porque eso genera conflictos internos en Hibernate. La solución fue diferir ese guardado hasta el momento exacto en que la transacción original termina de confirmarse con éxito, y además, hacer que ese guardado ocurra en una transacción completamente nueva e independiente. Esto garantiza que el registro de auditoría efectivamente quede grabado en la base de datos de forma confiable, sin importar en qué momento del ciclo de vida de la petición ocurra.

Manejo de errores

Todos los errores que puede producir la aplicación, ya sea por datos inválidos, recursos que no existen, credenciales incorrectas, o fallos inesperados, pasan por un manejador centralizado (GlobalExceptionHandler). En vez de que cada controlador tenga que lidiar con sus propios errores de forma individual, este componente intercepta cualquier excepción en cualquier parte del sistema y la convierte en una respuesta JSON con un formato consistente, indicando el código

de estado HTTP correspondiente, una descripción del error, un mensaje específico, la ruta donde ocurrió, y la fecha y hora exacta.

- **Ejemplo de token JWT y uso**

Que es un JWT

Un JSON Web Token (JWT) es una forma estándar de representar información de manera compacta y verificable entre el cliente y el servidor. Un token está formado por tres partes separadas por puntos: un encabezado, un cuerpo (payload) con la información real, y una firma que garantiza que nadie modificó el contenido después de que el servidor lo generó.

Qué contiene el payload en LogiTrack

Cuando el servidor genera un token, incluye dentro de este los siguientes datos:

- sub: el nombre de usuario de la persona autenticada.
- rol: el rol del usuario, ya sea ADMIN o EMPLEADO.
- iat: el momento exacto en que se generó el token.
- exp: el momento en que el token dejará de ser válido.

Por ejemplo, el payload de un token luce así:

```
{
  "sub": "admin",
  "rol": "ADMIN",
  "iat": 1753294400,
  "exp": 1755886400
}
```

Cómo se usa el token paso a paso

Primero, el usuario obtiene su token iniciando sesión:

POST /auth/login
Content-Type: application/json

```
{
  "username": "admin",
  "password": "Admin123!"
}
```

El servidor responde con el token, junto con algunos datos básicos del usuario:

```
{  
  "idUsuario": 1,  
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJhZG1pbils...",  
  "username": "admin",  
  "rol": "ADMIN"  
}
```

A partir de ese momento, cada vez que el cliente quiera acceder a un endpoint protegido, debe incluir ese token en el encabezado Authorization de la petición, precedido por la palabra Bearer:

GET /productos

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJhZG1pbils...

Cuando esa petición llega al servidor, el filtro de autenticación la intercepta antes de que llegue a cualquier controlador. Extrae el token del encabezado, verifica que la firma sea auténtica (es decir, que el token realmente fue generado por este servidor y no fue alterado), confirma que no haya expirado, y si todo está en orden, identifica al usuario y su rol, permitiendo que la petición continúe normalmente. Si algo falla en cualquiera de esas validaciones, el servidor responde con un error 401, indicando que la petición no está autorizada.

Finalmente, el token tiene una duración de 24 horas desde el momento en que fue generado. Una vez que expira, cualquier intento de usarlo será rechazado, y el usuario deberá volver a iniciar sesión para obtener un token nuevo.